Live Cloud Deployment (Task 1 and Task 2): https://zain-aic354-ml-systems.streamlit.app

One Streamlit app with two tabs: Titanic Survival Prediction and Heart Disease Prediction.

COMSATS University Islamabad

Lahore Campus

Department of Computer Science

A S S I G N M E N T

Assignment No. 2: Developing an End-to-End Machine Learning System

Course Title	Machine Learning Fundamentals
Course Code	AIC354
Submitted To	Dr. Rao Muhammad Adeel Nawab
Submitted By	Zain Aftab
Registration No.	FA24-BSE-155
Section	B
Semester	Fall 2026 (5th Semester)
Date of Submission	11 October 2026

Developing an End-to-End Machine Learning System

Assignment 2 (AIC354): Titanic Survival Prediction and Heart Disease Prediction

Developer	Zain Aftab
Registration No.	FA24-BSE-155
Section / Semester	B / Fall 2026 (5th Semester)
Program	BS Software Engineering, COMSATS University Islamabad, Lahore Campus
Course	Machine Learning Fundamentals (AIC354)
Instructor	Dr. Rao Muhammad Adeel Nawab
Base Code	Titanic Passenger Survival Prediction System by Mr. Mohsin Afzal and Dr. Rao Muhammad Adeel Nawab (GitHub repo)
IDE / Language	Jupyter Notebook / Python 3.11
Libraries	NumPy 2.2.6, Pandas 2.3.3, scikit-learn 1.7.2, Pickle (built-in), PrettyTable, Matplotlib, Seaborn, Streamlit 1.64.0
Submission Deadline	Sunday, 11 October 2026, 23:59

Aim of this Assignment

Learn how to build a **complete Machine Learning system**, from data collection to cloud deployment, using structured data. The instructor's Titanic Passenger Survival Prediction System is first run and understood, and then the **exact same steps** are reused to build a Heart Disease Prediction System on a real medical dataset from the UCI Machine Learning Repository.

Mapping of Assignment Tasks to this Notebook

Assignment Task	Where it is addressed
Task 1: Run and Understand the Titanic system	Part A, Steps 1 to 9 (instructor's code, executed, with an "Understanding" note for every step)
Task 1: Deploy the Titanic model	Part C, Step 10 (Titanic tab of the live app)
Task 2: Download the Heart Disease dataset from UCI	Part B, Step 2 (original `processed.cleveland.data` file)

Assignment Task	Where it is addressed
Task 2: Build the system following the exact same steps, using only 4 input attributes	Part B, Steps 1 to 9 (Step 4 selects the 4 attributes)
Task 2: Deploy the Heart Disease model	Part C, Step 10 (Heart Disease tab of the live app)
Submission: Live link at the very top	First cell of this notebook

The Four Phases of the Machine Learning Cycle

Phase	What happens	Steps in this notebook
Training Phase	The algorithm learns patterns from the training data and the model is saved	Step 6
Testing Phase	The saved model predicts unseen testing data and its accuracy is measured	Step 7
Application Phase	The model predicts a brand-new instance entered by a user	Step 8
Feedback Phase	Users and experts review predictions and improvements are planned	Step 9

How to Run this Notebook

1. Keep this notebook next to the `titanic/`, `heart_disease/` and `streamlit-app/` folders.
2. Install the packages: `pip install -r requirements.txt`
3. Click **Kernel → Restart & Run All**. Every CSV file and both trained models (`.pkl`) are created by the notebook itself.

Table of Contents

1. **Part A: Task 1, Titanic Passenger Survival Prediction System** (Steps 1 to 9)
2. **Part B: Task 2, Heart Disease Prediction System** (Steps 1 to 9)
3. **Part C: Step 10, Deploying Both Models to the Cloud**
4. **Conclusion and References**

Environment Setup

The next cell gives Jupyter a formal look (Times New Roman, navy headings). It does not change any results.

```
# Formal look: Times New Roman, navy #1B2A49, double-rule headings (used by
Jupyter; harmless elsewhere)
from IPython.display import HTML, display
display(HTML("""<style>
h1, h2 {font-family:'Times New Roman',serif; color:#1B2A49; border-bottom:3px
double #1B2A49;}
h3, h4 {font-family:'Times New Roman',serif; color:#1B2A49;}
.jp-RenderedHTMLCommon th, .rendered_html th {background:#1B2A49 !important;
color:#fff !important;}
</style>"""))
print("Styling applied.")
```

```
Styling applied.
```

PART A: Task 1, Titanic Passenger Survival Prediction System

Task 1 (Run and Understand): the instructor's *Titanic_Passenger_Survival_Recommended.ipynb* code is executed below exactly as provided (only the file paths point to the `titanic/` folder). Under every step, an **Understanding** note explains in simple words what the step does and why it is needed.

Problem Definition

A **binary classification** task: predict whether a passenger **survived (Yes)** or **did not survive (No)** the Titanic disaster from four passenger attributes. The dataset is a simplified 100-row sample.

Input and Output

Inputs (Features / Attributes)

Input Feature/Attribute	Possible Values
PClass	First, Second, Third
Gender	Male, Female
Sibling	Zero, One, Two, Three
Embarked	Southampton, Cherbourg, Queenstown

Output (Prediction)

Output Attribute	Possible Values
Survived	Yes, No

Examples (Titanic Dataset)

PClass	Gender	Sibling	Embarked	Survived
Third	Male	One	Southampton	No
Second	Female	Zero	Southampton	Yes
Third	Male	Zero	Southampton	No
Third	Female	Three	Southampton	Yes
Third	Male	Zero	Queenstown	No

Step 1: Import Libraries

> **Understanding:** All tools are loaded first: **Pandas** for tables, **NumPy** for arrays, **LabelEncoder** to turn text into numbers, **SVC** (Support Vector Classifier) as the learning algorithm, **pickle** to save the model, and metrics to measure how good the model is.

```
"""
Purpose of this code:
----------------------
This script imports the required libraries for building and evaluating a
machine learning model. It sets up tools for data manipulation, preprocessing,
splitting datasets, training a Support Vector Machine (SVM) model,
evaluating accuracy, and displaying results in tabular form.
"""

# Import numerical computing library (used for arrays, math operations, etc.)
import numpy as np

# Import data manipulation library (used for DataFrames, reading/writing data,
etc.)
import pandas as pd

# Import pickle (used for saving and loading Python objects/models)
import pickle

# Import train_test_split (used to divide dataset into training and testing sets)
from sklearn.model_selection import train_test_split

# Import LabelEncoder (used to convert categorical values into numeric codes)
from sklearn.preprocessing import LabelEncoder

# Import Support Vector Machine (SVM) model from scikit-learn
from sklearn import svm

# Import evaluation metrics
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

# Import PrettyTable (used to display results in a clean, table format)
from prettytable import PrettyTable
```

```
# Import plotting libraries (used for the confusion matrix heatmap)
import matplotlib.pyplot as plt
import seaborn as sns
```

Step 2: Load Sample Data

> **Understanding:** The 100-row sample dataset is read from a CSV file into a DataFrame. Every row is one passenger; the last column (`Survived`) is the output we want to predict.

```
"""
Purpose of this section:
-------------------------
This part of the code loads a sample dataset from a CSV file into a
pandas DataFrame. It then displays the dataset so that students can
see the raw data being used for further processing and model training.
"""

# Read the CSV file and load it into a pandas DataFrame
# 'titanic/sample-data.csv' should be present in the working directory
sample_data = pd.read_csv("titanic/sample-data.csv")

# Print a header to indicate that sample data is being displayed
print("\n\nSample Data:")
print("============\n")

# Configure pandas to display all rows and columns (so nothing gets hidden with
'...')
pd.set_option("display.max_rows", None, "display.max_columns", None)

# Print the complete dataset
print(sample_data)
```

Sample Data:
============

	PClass	Gender	Sibling	Embarked	Survived
0	Third	Male	One	Southampton	No
1	Second	Female	Zero	Southampton	Yes
2	Third	Male	Zero	Southampton	No
3	Third	Female	Three	Southampton	Yes
4	Third	Male	Zero	Queenstown	No
5	First	Female	Three	Southampton	Yes
6	Third	Male	Zero	Southampton	No
7	Third	Male	Zero	Southampton	Yes
8	First	Male	Zero	Southampton	No
9	Second	Male	Zero	Southampton	Yes
10	Third	Male	One	Queenstown	No
11	First	Male	Zero	Cherbourg	Yes
12	First	Male	Zero	Cherbourg	No
13	Second	Female	Zero	Southampton	Yes
14	Second	Male	Zero	Southampton	No
15	Third	Male	One	Cherbourg	Yes
16	Third	Male	Two	Cherbourg	No
17	Third	Male	Zero	Southampton	Yes
18	Third	Male	Zero	Southampton	No
19	Third	Female	One	Cherbourg	Yes
20	Third	Male	Zero	Cherbourg	No
21	First	Female	Zero	Southampton	Yes
22	First	Male	Zero	Cherbourg	No
23	Second	Female	Zero	Southampton	Yes
24	Third	Female	One	Southampton	No
25	Third	Female	Zero	Southampton	Yes
26	Third	Male	Zero	Southampton	No
27	Third	Male	Zero	Southampton	Yes
28	Third	Male	Zero	Southampton	No
29	Third	Female	One	Queenstown	Yes
30	Third	Female	Two	Southampton	No
31	First	Female	Zero	Cherbourg	Yes
32	Third	Male	Two	Southampton	No
33	First	Female	Zero	Southampton	Yes
34	First	Male	One	Cherbourg	No
35	First	Female	Zero	Southampton	Yes
36	First	Male	One	Southampton	No
37	Third	Female	One	Queenstown	Yes
38	Third	Female	One	Southampton	No
39	Second	Female	One	Southampton	Yes
40	Second	Female	One	Southampton	No
41	Third	Female	Zero	Queenstown	Yes
42	Third	Male	Zero	Cherbourg	No
43	First	Female	Zero	Cherbourg	Yes
44	Third	Male	Three	Southampton	No
45	Third	Female	One	Southampton	Yes
46	Third	Male	Zero	Southampton	No
47	Third	Male	Zero	Southampton	Yes
48	Third	Male	One	Southampton	No
49	Second	Female	Zero	Southampton	Yes
50	First	Male	Three	Southampton	No
51	Third	Male	Zero	Southampton	Yes
52	Third	Female	One	Southampton	No
53	Second	Female	Zero	Southampton	Yes

```
54   Third  Female   Three  Southampton       No
55   Third  Female    Zero  Southampton      Yes
56   Third  Female    Zero  Southampton       No
57   Third  Female     One  Southampton      Yes
58   Third    Male    Zero    Cherbourg       No
59   Third    Male    Zero  Southampton      Yes
60  Second    Male    Zero  Southampton       No
61   First  Female     One  Southampton      Yes
62   Third    Male   Three   Queenstown       No
63   Third  Female    Zero   Queenstown      Yes
64  Second  Female    Zero  Southampton       No
65   Third  Female    Zero    Cherbourg      Yes
66   Third    Male    Zero  Southampton       No
67  Second    Male    Zero  Southampton      Yes
68   Third    Male    Zero  Southampton       No
69   First    Male    Zero  Southampton      Yes
70   Third    Male    Zero  Southampton       No
71  Second    Male     Two  Southampton      Yes
72   Third    Male    Zero    Cherbourg       No
73   Third  Female    Zero  Southampton      Yes
74   Third  Female    Zero  Southampton       No
75   First  Female     One  Southampton      Yes
76   Third    Male     One  Southampton       No
77  Second  Female    Zero  Southampton      Yes
78   Third    Male    Zero  Southampton       No
79   Third  Female     Two  Southampton      Yes
80   Third    Male     Two  Southampton       No
81   Third  Female    Zero  Southampton      Yes
82   Third    Male    Zero  Southampton       No
83   First    Male     One  Southampton      Yes
84  Second    Male    Zero  Southampton       No
85   Third  Female     One   Queenstown      Yes
86  Second    Male    Zero  Southampton       No
87   First  Female     One  Southampton      Yes
88   Third    Male    Zero  Southampton       No
89  Second  Female    Zero  Southampton      Yes
90  Second    Male    Zero  Southampton       No
91  Second  Female    Zero  Southampton      Yes
92   Third    Male    Zero    Cherbourg       No
93   First    Male     One  Southampton      Yes
94   First    Male     One  Southampton       No
95   Third    Male     One    Cherbourg      Yes
96   Third    Male     One   Queenstown       No
97   Third    Male     One  Southampton      Yes
98  Second    Male     One  Southampton       No
99   Third    Male     One  Southampton      Yes
```

Step 3: Understand and Pre-process Sample Data

Step 3.1: Understand Sample Data

> **Understanding:** Before building anything we check *what* the data contains: 5 attributes (4 inputs + 1 output) and 100 instances.

```
"""
Purpose of this section:
------------------------
This part of the code helps us understand the dataset by:
1. Printing the names of all attributes (columns) in the DataFrame.
2. Showing the total number of instances (rows) available in the dataset.
"""

# Print a header for clarity
print("\n\nAttributes in Sample Data:")
print("==========================\n")

# Display all column names (attributes) in the dataset
print(sample_data.columns)

# Print a header for instance count
print("\n\nNumber of Instances in Sample Data:", sample_data["PClass"].count())
print("=======================================\n")
```

```
Attributes in Sample Data:
==========================

Index(['PClass', 'Gender', 'Sibling', 'Embarked', 'Survived'], dtype='object')


Number of Instances in Sample Data: 100
=======================================
```

Step 3.2: Pre-process Sample Data

- Sample Data is already Preprocessed
- No Preprocessing needs to be Performed

> **Understanding:** Pre-processing means cleaning the data (missing values, wrong types, inconsistent text). This sample was prepared in advance, so no cleaning is needed here. In Part B real cleaning is required.

Step 4: Feature Extraction

- Features are already Extracted
- No Feature Extraction needs to be Performed

> **Understanding:** Feature extraction means choosing or creating the attributes the model will learn from. The four input attributes are already selected.

Step 5: Label Encoding the Sample Data (Input and Output is converted in Numeric Representation)

Step 5.1: Train the Label Encoder

> **Understanding:** ML algorithms only understand numbers. A **LabelEncoder** is trained on the list of possible values of each attribute and gives each value a number in **alphabetical order** (for example `Cherbourg = 0, Queenstown = 1, Southampton = 2`).

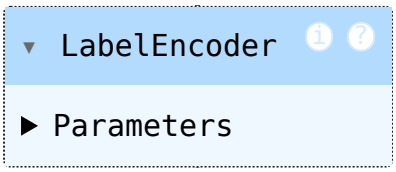
```
"""
Purpose of this section:
-------------------------
This part of the code sets up and trains Label Encoders.
Label Encoding is used to convert categorical values (like 'Male', 'Female',
'First', 'Second')
into numeric codes that machine learning algorithms can work with.

Steps:
1. Define sample label categories for each attribute.
2. Initialize a LabelEncoder for each attribute.
3. Train (fit) each LabelEncoder with its respective category values.
"""

# Define label categories for each feature
# These are possible values each column can have in the dataset
pclass = pd.DataFrame({"Pclass": ["First", "Second", "Third"]})
gender = pd.DataFrame({"Gender": ["Male", "Female"]})
sibling = pd.DataFrame({"Sibling": ["Zero", "One", "Two", "Three"]})
embarked = pd.DataFrame({"Embarked": ["Southampton", "Cherbourg", "Queenstown"]})
survived = pd.DataFrame({"Survived": ["Yes", "No"]})

# Create LabelEncoder objects for each categorical column
pclass_label_encoder = LabelEncoder()
gender_label_encoder = LabelEncoder()
sibling_label_encoder = LabelEncoder()
embarked_label_encoder = LabelEncoder()
survived_label_encoder = LabelEncoder()

# Train (fit) the encoders on the given categories
# np.ravel() flattens the DataFrame column into a 1D array for the encoder
pclass_label_encoder.fit(np.ravel(pclass))
gender_label_encoder.fit(np.ravel(gender))
sibling_label_encoder.fit(np.ravel(sibling))
embarked_label_encoder.fit(np.ravel(embarked))
survived_label_encoder.fit(np.ravel(survived))
```



Step 5.2: Label Encode the Output

> **Understanding:** The output `Survived` is converted: `No = 0`, `Yes = 1`.

```
"""
Purpose of this section:
-------------------------
This part of the code applies Label Encoding to the target/output variable
("Survived") in the dataset. The categorical values ("Yes", "No") are
converted into numerical values (e.g., 1 and 0) that can be used by
machine learning algorithms.
"""

# Create copies of the dataset (one for encoded output, one to keep original)
sample_data_encoded_output = sample_data.copy()
original_sample_data = sample_data.copy()

# Transform "Survived" (categorical -> numeric) using the trained LabelEncoder
print("\n\nSurvived Attribute After Label Encoding:")
print("=======================================\n")
sample_data["encoded_survived"] =
survived_label_encoder.transform(sample_data['Survived'])

# Show original "Survived" values and their numeric encodings
print(sample_data[["Survived", "encoded_survived"]])

# Replace original categorical columns with encoded values for the output dataset
sample_data_encoded_output[['PClass', 'Gender', 'Sibling', 'Embarked',
'Survived']] = \
    sample_data[['PClass', 'Gender', 'Sibling', 'Embarked', 'encoded_survived']]

# Save the encoded dataset to a CSV file (useful for training/testing later)
sample_data_encoded_output.to_csv(r'titanic/sample-data-encoded-output.csv',
index=False, header=True)
print("\nSaved: titanic/sample-data-encoded-output.csv")
```

Survived Attribute After Label Encoding:
==========

	Survived	encoded_survived
0	No	0
1	Yes	1
2	No	0
3	Yes	1
4	No	0
5	Yes	1
6	No	0
7	Yes	1
8	No	0
9	Yes	1
10	No	0
11	Yes	1
12	No	0
13	Yes	1
14	No	0
15	Yes	1
16	No	0
17	Yes	1
18	No	0
19	Yes	1
20	No	0
21	Yes	1
22	No	0
23	Yes	1
24	No	0
25	Yes	1
26	No	0
27	Yes	1
28	No	0
29	Yes	1
30	No	0
31	Yes	1
32	No	0
33	Yes	1
34	No	0
35	Yes	1
36	No	0
37	Yes	1
38	No	0
39	Yes	1
40	No	0
41	Yes	1
42	No	0
43	Yes	1
44	No	0
45	Yes	1
46	No	0
47	Yes	1
48	No	0
49	Yes	1
50	No	0
51	Yes	1
52	No	0
53	Yes	1

```
54       No                0
55      Yes                1
56       No                0
57      Yes                1
58       No                0
59      Yes                1
60       No                0
61      Yes                1
62       No                0
63      Yes                1
64       No                0
65      Yes                1
66       No                0
67      Yes                1
68       No                0
69      Yes                1
70       No                0
71      Yes                1
72       No                0
73      Yes                1
74       No                0
75      Yes                1
76       No                0
77      Yes                1
78       No                0
79      Yes                1
80       No                0
81      Yes                1
82       No                0
83      Yes                1
84       No                0
85      Yes                1
86       No                0
87      Yes                1
88       No                0
89      Yes                1
90       No                0
91      Yes                1
92       No                0
93      Yes                1
94       No                0
95      Yes                1
96       No                0
97      Yes                1
98       No                0
99      Yes                1

Saved: titanic/sample-data-encoded-output.csv
```

Step 5.3: Label Encode the Input

> **Understanding:** The four input attributes are converted with their own encoders. The fully numeric table is saved as `sample-data-encoded.csv`.

```
"""
Purpose of this section:
-------------------------
```

```
This part of the code applies Label Encoding to the input features
("PClass", "Gender", "Sibling", "Embarked"). Each categorical value
is converted into a numeric value so that machine learning models
can process them.
"""

# Create copies of the dataset (to keep original values safe)
sample_data_encoded = sample_data_encoded_output.copy()

# Encode each input attribute
sample_data_encoded_output["encoded_pclass"] =
pclass_label_encoder.transform(sample_data_encoded_output['PClass'])
sample_data_encoded_output["encoded_gender"] =
gender_label_encoder.transform(sample_data_encoded_output['Gender'])
sample_data_encoded_output["encoded_sibling"] =
sibling_label_encoder.transform(sample_data_encoded_output['Sibling'])
sample_data_encoded_output["encoded_embarked"] =
embarked_label_encoder.transform(sample_data_encoded_output['Embarked'])

# Replace categorical columns with encoded values in the dataset
sample_data_encoded[['PClass', 'Gender', 'Sibling', 'Embarked', 'Survived']] = \
    sample_data_encoded_output[['encoded_pclass', 'encoded_gender',
'encoded_sibling', 'encoded_embarked', 'Survived']]

# Print encoded dataset (with numeric values instead of categories)
print("\n\nSample Data after Label Encoding:")
print("=================================\n")
print(sample_data_encoded)

# Save the encoded dataset to a new CSV file
sample_data_encoded.to_csv(r'titanic/sample-data-encoded.csv', index=False,
header=True)
```

Sample Data after Label Encoding:
==========

	PClass	Gender	Sibling	Embarked	Survived
0	2	1	0	2	0
1	1	0	3	2	1
2	2	1	3	2	0
3	2	0	1	2	1
4	2	1	3	1	0
5	0	0	1	2	1
6	2	1	3	2	0
7	2	1	3	2	1
8	0	1	3	2	0
9	1	1	3	2	1
10	2	1	0	1	0
11	0	1	3	0	1
12	0	1	3	0	0
13	1	0	3	2	1
14	1	1	3	2	0
15	2	1	0	0	1
16	2	1	2	0	0
17	2	1	3	2	1
18	2	1	3	2	0
19	2	0	0	0	1
20	2	1	3	0	0
21	0	0	3	2	1
22	0	1	3	0	0
23	1	0	3	2	1
24	2	0	0	2	0
25	2	0	3	2	1
26	2	1	3	2	0
27	2	1	3	2	1
28	2	1	3	2	0
29	2	0	0	1	1
30	2	0	2	2	0
31	0	0	3	0	1
32	2	1	2	2	0
33	0	0	3	2	1
34	0	1	0	0	0
35	0	0	3	2	1
36	0	1	0	2	0
37	2	0	0	1	1
38	2	0	0	2	0
39	1	0	0	2	1
40	1	0	0	2	0
41	2	0	3	1	1
42	2	1	3	0	0
43	0	0	3	0	1
44	2	1	1	2	0
45	2	0	0	2	1
46	2	1	3	2	0
47	2	1	3	2	1
48	2	1	0	2	0
49	1	0	3	2	1
50	0	1	1	2	0
51	2	1	3	2	1
52	2	0	0	2	0
53	1	0	3	2	1

54	2	0	1	2	0
55	2	0	3	2	1
56	2	0	3	2	0
57	2	0	0	2	1
58	2	1	3	0	0
59	2	1	3	2	1
60	1	1	3	2	0
61	0	0	0	2	1
62	2	1	1	1	0
63	2	0	3	1	1
64	1	0	3	2	0
65	2	0	3	0	1
66	2	1	3	2	0
67	1	1	3	2	1
68	2	1	3	2	0
69	0	1	3	2	1
70	2	1	3	2	0
71	1	1	2	2	1
72	2	1	3	0	0
73	2	0	3	2	1
74	2	0	3	2	0
75	0	0	0	2	1
76	2	1	0	2	0
77	1	0	3	2	1
78	2	1	3	2	0
79	2	0	2	2	1
80	2	1	2	2	0
81	2	0	3	2	1
82	2	1	3	2	0
83	0	1	0	2	1
84	1	1	3	2	0
85	2	0	0	1	1
86	1	1	3	2	0
87	0	0	0	2	1
88	2	1	3	2	0
89	1	0	3	2	1
90	1	1	3	2	0
91	1	0	3	2	1
92	2	1	3	0	0
93	0	1	0	2	1
94	0	1	0	2	0
95	2	1	0	0	1
96	2	1	0	1	0
97	2	1	0	2	1
98	1	1	0	2	0
99	2	1	0	2	1

Step 6: Execute the Training Phase

Step 6.1: Splitting Sample Data into Training Data and Testing Data

> **Understanding: Training Phase begins.** The data is split into **80% training data** (80 passengers, used to learn) and **20% testing data** (20 passengers, kept hidden to check the model later). `shuffle=False` keeps the original row order, so the split is the same every time.

```
"""
Purpose of this section:
-------------------------
This part of the code splits the dataset into Training Data and Testing Data.
- Training Data is used to teach the machine learning model.
- Testing Data is used to check how well the model performs on unseen data.
"""

# Split dataset into training (80%) and testing (20%)
# random_state=0 ensures reproducibility (same split every time)
# shuffle=False keeps the original order of rows (no shuffling before splitting)
training_data_encoded, testing_data_encoded = train_test_split(
    sample_data_encoded,
    test_size=0.2,
    random_state=0,
    shuffle=False
)

# Save the training and testing datasets to CSV files
training_data_encoded.to_csv(r'titanic/training-data-encoded.csv', index=False,
header=True)
testing_data_encoded.to_csv(r'titanic/testing-data-encoded.csv', index=False,
header=True)

print("Training Data shape:", training_data_encoded.shape)
print("Testing Data shape:", testing_data_encoded.shape)
```

```
Training Data shape: (80, 5)
Testing Data shape: (20, 5)
```

Step 6.2: Splitting Input Vectors and Outputs / Labels of Training Data

> **Understanding:** The training table is divided into **input vectors** (first 4 columns, `X`) and **output labels** (last column, `y`).

```
"""
Purpose of this section:
-------------------------
This part of the code separates the features (input vectors) and
the labels (output values) from the training dataset.
"""

# Extract input vectors (features) from training data
input_vector_train = training_data_encoded.iloc[:, :-1]

# Extract output labels (target variable) from training data
output_label_train = training_data_encoded.iloc[:, -1]

print("Input vectors (train) shape:", input_vector_train.shape)
print("Output labels (train) shape:", output_label_train.shape)
```

```
Input vectors (train) shape: (80, 4)
Output labels (train) shape: (80,)
```

Step 6.3: Train the Support Vector Classifier

> **Understanding:** The **Support Vector Classifier** finds the boundary that best separates survivors from non-survivors. `fit()` is the actual learning step.

```
"""
Purpose of this section:
------------------------
This part of the code trains a Support Vector Classifier (SVC) on the training
data.
The SVC is a type of machine learning model that finds the best boundary
to separate different classes (e.g., survived vs. not survived).
"""

print("\n\nTraining the Support Vector Classifier on Training Data")
print("============================================================\n")

# Initialize the Support Vector Classifier with chosen parameters
svc_model = svm.SVC(gamma='auto', random_state=0)

# Train the model using the training data (features and labels)
svc_model.fit(input_vector_train, np.ravel(output_label_train))

# Print trained model details (parameters and values)
print(svc_model)
```

```
Training the Support Vector Classifier on Training Data
============================================================

SVC(gamma='auto', random_state=0)
```

Step 6.4: Save the Trained Model

> **Understanding:** The trained model is saved to a `.pkl` file, so it can be reused (testing, application, deployment) without retraining.

```
"""
Purpose of this section:
------------------------
This part of the code saves the trained Support Vector Classifier (SVC)
model to your computer so it can be reused later without retraining.
"""

# Save the trained SVC model to a file named "svc_trained_model.pkl"
pickle.dump(svc_model, open('titanic/svc_trained_model.pkl', 'wb'))
print("Saved: titanic/svc_trained_model.pkl")
```

```
Saved: titanic/svc_trained_model.pkl
```

Step 7: Execute the Testing Phase

Step 7.1: Splitting Input Vectors and Outputs/Labels of Testing Data

> **Understanding: Testing Phase begins.** The 20 testing passengers are split into inputs and true labels, exactly like the training data.

```
"""
Purpose of this section:
-------------------------
This part of the code separates the features (inputs) and the labels (outputs)
from the testing dataset.
"""

input_vector_test = testing_data_encoded.iloc[:, :-1]
output_label_test = testing_data_encoded.iloc[:, -1]

print("Input vectors (test) shape:", input_vector_test.shape)
print("Output labels (test) shape:", output_label_test.shape)
```

```
Input vectors (test) shape: (20, 4)
Output labels (test) shape: (20,)
```

Step 7.2: Load the Saved Model

> **Understanding:** The saved model is loaded back from disk, which proves the saved file works.

```
"""
Purpose of this section:
-------------------------
This part of the code loads the previously saved Support Vector Classifier (SVC)
model from disk into memory.
"""

model = pickle.load(open('titanic/svc_trained_model.pkl', 'rb'))
print("Model loaded from titanic/svc_trained_model.pkl")
```

```
Model loaded from titanic/svc_trained_model.pkl
```

Step 7.3: Evaluate the Machine Learning Model

Step 7.3.1: Make Predictions with the Trained Models on Testing Data

> **Understanding:** The model predicts the 20 unseen passengers. The `Predictions` column is placed next to the true `Survived` column for comparison.

```
"""
Purpose of this section:
-------------------------
This part of the code evaluates the trained Support Vector Classifier (SVC)
on the testing data.
"""

model_predictions = model.predict(input_vector_test)

pd.options.mode.chained_assignment = None
```

```
testing_data_encoded["Predictions"] = model_predictions
testing_data_encoded.to_csv(r'titanic/model-predictions.csv', index=False,
header=True)

model_predictions = testing_data_encoded
print("\n\nPredictions Returned by svc_trained_model:")
print("=========================================\n")
print(model_predictions)
```

```
Predictions Returned by svc_trained_model:
==========================================

    PClass  Gender  Sibling  Embarked  Survived  Predictions
80       2       1        2         2         0            0
81       2       0        3         2         1            1
82       2       1        3         2         0            0
83       0       1        0         2         1            0
84       1       1        3         2         0            0
85       2       0        0         1         1            1
86       1       1        3         2         0            0
87       0       0        0         2         1            1
88       2       1        3         2         0            0
89       1       0        3         2         1            1
90       1       1        3         2         0            0
91       1       0        3         2         1            1
92       2       1        3         0         0            0
93       0       1        0         2         1            0
94       0       1        0         2         0            0
95       2       1        0         0         1            1
96       2       1        0         1         0            0
97       2       1        0         2         1            0
98       1       1        0         2         0            0
99       2       1        0         2         1            0
```

Step 7.4: Calculate the Accuracy Score

> **Understanding: Accuracy** = correct predictions / total predictions. An accuracy of 0.80 means 16 of the 20 testing passengers were predicted correctly.

```
"""
Purpose of this section:
------------------------
This part of the code calculates how accurate the trained model is on the
testing dataset.
"""

model_accuracy_score = accuracy_score(
    model_predictions["Survived"],
    model_predictions["Predictions"]
)

print("\n\nAccuracy Score:")
print("===============\n")
print(round(model_accuracy_score, 2))
```

```
Accuracy Score:
===============

0.8
```

Step 7.5: Detailed Evaluation Metrics (Precision, Recall, F1, Confusion Matrix)

> **Understanding:** Accuracy alone can hide errors. **Precision**, **Recall**, **F1-score** and the **Confusion Matrix** show exactly which class the model gets wrong.

```
"""
Purpose of this section:
-------------------------
Accuracy alone can be misleading, especially on small or imbalanced
datasets. This section adds the standard classification metrics:
- Precision: of everyone the model predicted as "Survived", how many actually
survived?
- Recall: of everyone who actually survived, how many did the model catch?
- F1-score: the harmonic mean of precision and recall.
- Confusion Matrix: a table of correct vs. incorrect predictions per class.
"""

print("\nClassification Report:")
print("======================\n")
print(classification_report(
    model_predictions["Survived"],
    model_predictions["Predictions"],
    target_names=["Not Survived (0)", "Survived (1)"]
))

conf_matrix = confusion_matrix(model_predictions["Survived"],
model_predictions["Predictions"])
conf_matrix_df = pd.DataFrame(
    conf_matrix,
    index=["Actual: Not Survived", "Actual: Survived"],
    columns=["Predicted: Not Survived", "Predicted: Survived"]
)
print("\nConfusion Matrix:")
print("=================\n")
print(conf_matrix_df)

plt.figure(figsize=(5, 4))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Not Survived', 'Survived'],
            yticklabels=['Not Survived', 'Survived'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix - SVC on Titanic Testing Data')
plt.tight_layout()
plt.show()
```

```
Classification Report:
======================

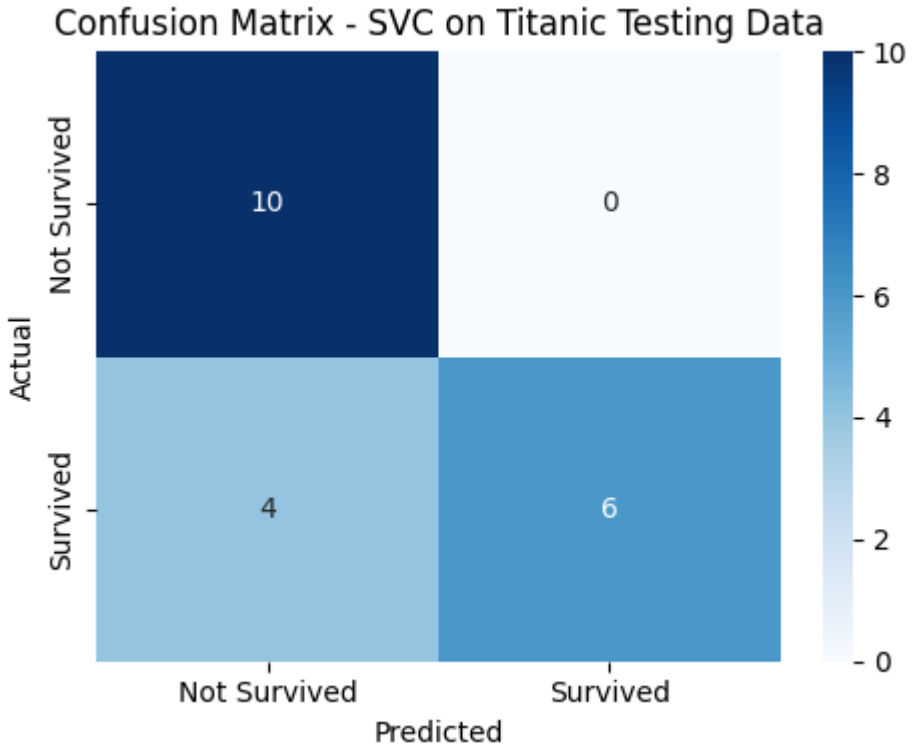
                  precision    recall  f1-score   support

Not Survived (0)       0.71      1.00      0.83        10
    Survived (1)       1.00      0.60      0.75        10

        accuracy                           0.80        20
       macro avg       0.86      0.80      0.79        20
    weighted avg       0.86      0.80      0.79        20


Confusion Matrix:
=================

                     Predicted: Not Survived  Predicted: Survived
Actual: Not Survived                       10                    0
Actual: Survived                            4                    6
```



Step 8: Execute the Application Phase

Step 8.1: Take Input from User

> **Understanding: Application Phase begins.** A new passenger that the model has never seen is entered (pre-filled here so the notebook runs from top to bottom).

```
"""
Purpose of this section:
------------------------
This part of the code allows the user to enter their own data
(PClass, Gender, Sibling, Embarked). In an interactive Jupyter session,
```

```
running this cell will prompt you for input(). For this notebook copy,
we pre-fill a demo passenger so the notebook can also be read/run
top-to-bottom without needing to type anything.

To try your own values interactively, replace the four lines below with:
    pclass_input = input("Please enter PClass here (First, Second, Third):
").strip()
    gender_input = input("Please enter your Gender here (Male, Female): ").strip()
    sibling_input = input("Please enter your Sibling here (Zero, One, Two, Three):
").strip()
    embarked_input = input("Please enter Embarked here (Cherbourg, Southampton,
Queenstown): ").strip()
"""

# Demo passenger (edit these values, or swap in input() calls as shown above)
pclass_input = "First"
gender_input = "Male"
sibling_input = "Zero"
embarked_input = "Queenstown"

print(f"PClass: {pclass_input}, Gender: {gender_input}, Sibling: {sibling_input},
Embarked: {embarked_input}")
```

```
PClass: First, Gender: Male, Sibling: Zero, Embarked: Queenstown
```

Step 8.2: Convert User Input into Feature Vector (Exactly Same as Feature Vectors of Sample Data)

> **Understanding:** The raw input is arranged into a one-row DataFrame with the **same column names and order** as the training data.

```
"""
Purpose of this section:
------------------------
This part of the code takes the user's input (entered as text)
and converts it into a structured feature vector (DataFrame).
"""

user_input = pd.DataFrame({
    'PClass': [pclass_input],
    'Gender': [gender_input],
    'Sibling': [sibling_input],
    'Embarked': [embarked_input]
})

print("\n\nUser Input Feature Vector:")
print("==========================\n")
print(user_input)
```

```
User Input Feature Vector:
==========================

  PClass Gender Sibling    Embarked
0  First   Male    Zero  Queenstown
```

Step 8.3: Label Encoding of Feature Vector (Exactly Same as Label Encoded Feature Vectors of Sample Data)

> **Understanding:** The input is encoded with the **same trained encoders**. Using different encoders would give the model wrong numbers.

```
"""
Purpose of this section:
-------------------------
This part of the code encodes the user's input into numeric values
using the previously trained LabelEncoders.
"""

unseen_data_features = user_input.copy()

unseen_data_features["PClass"] =
pclass_label_encoder.transform(user_input['PClass'])
unseen_data_features["Gender"] =
gender_label_encoder.transform(user_input['Gender'])
unseen_data_features["Sibling"] =
sibling_label_encoder.transform(user_input['Sibling'])
unseen_data_features["Embarked"] =
embarked_label_encoder.transform(user_input['Embarked'])

print("\n\nUser Input Encoded Feature Vector:")
print("==================================\n")
print(unseen_data_features)
```

```
User Input Encoded Feature Vector:
==================================

   PClass  Gender  Sibling  Embarked
0       0       1        3         1
```

Step 8.4: Load the Saved Model

> **Understanding:** The saved model is loaded, just as a real application would do.

```
"""
Purpose of this section:
-------------------------
This part of the code loads the previously saved Support Vector Classifier (SVC)
model from the .pkl file into memory for the Application Phase.
"""

model = pickle.load(open('titanic/svc_trained_model.pkl', 'rb'))
```

Step 8.5: Model Prediction

Step 8.5.1: Apply Model on the Label Encoded Feature Vector of unseen instance and return Prediction to the User

> **Understanding:** The model returns `1` (Survived) or `0` (Not Survived), which is converted back into readable text.

```
"""
Purpose of this section:
-------------------------
This part of the code uses the trained Support Vector Classifier (SVC)
to predict whether a new passenger (user input) survived or not.
"""

predicted_survival = model.predict(unseen_data_features)

if predicted_survival == 1:
    prediction = "SURVIVED"
if predicted_survival == 0:
    prediction = "NOT SURVIVED"

pretty_table = PrettyTable()
pretty_table.add_column("       ** Prediction **       ", [prediction])

print(pretty_table)
```

```
+--------------------------------+
|        ** Prediction **        |
+--------------------------------+
|            SURVIVED            |
+--------------------------------+
```

Step 9: Execute the Feedback Phase

Step 9.1: Collect Feedback from Users and Domain Experts on Performance of the Model Deployed in the Real World

> **Understanding: Feedback Phase begins.** After deployment, users and domain experts check whether the predictions make sense in the real world.

```
"""
Purpose of this section:
-------------------------
Step 9.1: Example feedback collected from a domain expert reviewing
this model's predictions on a few real-world-style cases.
"""

example_feedback = pd.DataFrame({
    "Reviewer": ["Domain Expert (History)", "Student Tester", "Student Tester"],
    "Case": [
        "Third class, male, Southampton -> predicted NOT SURVIVED",
        "First class, female -> predicted SURVIVED",
        "Third class, male, large family (3 siblings) -> predicted SURVIVED
(unexpected)",
    ],
    "Feedback": [
        "Matches historical record: third-class men had the lowest survival
```

```
rate.",
        "Matches historical record: women and first-class passengers were
prioritized.",
        "Surprising - the model may be picking up a spurious pattern from the
small (100-row) sample rather than a real effect. Needs more data to confirm.",
    ],
})
print(example_feedback.to_string(index=False))
```

```
               Reviewer
Case
Feedback
Domain Expert (History)                         Third class, male, Southampton -> pr
edicted NOT SURVIVED
Matches historical record: third-class men had the lowest survival rate.
         Student Tester                                          First class, female -
> predicted SURVIVED
Matches historical record: women and first-class passengers were prioritized.
         Student Tester Third class, male, large family (3 siblings) -> predicted S
URVIVED (unexpected) Surprising - the model may be picking up a spurious pattern fr
om the small (100-row) sample rather than a real effect. Needs more data to confir
m.
```

Step 9.2: Make a List of Potential Improvements

> **Understanding:** Problems found in the feedback are turned into a list of concrete improvements.

```
"""
Purpose of this section:
-------------------------
Step 9.2: Potential improvements identified from the feedback above.
"""

improvements = [
    "Use the full Titanic dataset (~900 passengers) instead of the 100-row toy
sample, so rare combinations (e.g. large families) are better represented",
    "Add Age and Fare as numeric features - both are known strong predictors of
survival",
    "Try class-balanced or cross-validated evaluation, since the toy dataset may
not be representative",
    "Compare additional models (e.g. Random Forest, Gradient Boosting) beyond
SVC/Logistic Regression/Decision Tree",
]
for i, item in enumerate(improvements, 1):
    print(f"{i}. {item}")
```

```
1. Use the full Titanic dataset (~900 passengers) instead of the 100-row toy sampl
e, so rare combinations (e.g. large families) are better represented
2. Add Age and Fare as numeric features - both are known strong predictors of survi
val
3. Try class-balanced or cross-validated evaluation, since the toy dataset may not
be representative
4. Compare additional models (e.g. Random Forest, Gradient Boosting) beyond SVC/Log
istic Regression/Decision Tree
```

Step 9.3: Improve the Model Based on Feedback

> **Understanding:** The improvements lead to a new version of the model, which must be retrained and retested before it replaces the deployed one.

```
"""
Purpose of this section:
-------------------------
Step 9.3: In a real deployment, the next version of this notebook would
retrain on the improvements listed above and re-run Steps 6-7 to confirm
the new model's accuracy improves before replacing the deployed model.
"""

print("Next version: retrain with Age + Fare features on the full dataset, then
re-evaluate before replacing svc_trained_model.pkl")
```

```
Next version: retrain with Age + Fare features on the full dataset, then re-evaluat
e before replacing svc_trained_model.pkl
```

Task 1 Result

The instructor's system ran from top to bottom without errors. The Support Vector Classifier reached an **accuracy of 0.80** on the 20 held-out passengers, and the demo passenger (First class, Male, no siblings, Queenstown) was predicted as **SURVIVED**. The trained model `titanic/svc_trained_model.pkl` is deployed in Part C.

PART B: Task 2, Heart Disease Prediction System

Task 2: the Titanic code above is modified to build a **Heart Disease Prediction System** that follows the **exact same steps** (Steps 1 to 9). As required, only **4 input attributes** are used.

Problem Definition

A **binary classification** task: predict whether a patient **has heart disease (Yes)** or **does not (No)** from four clinical attributes.

Dataset

Source	UCI Machine Learning Repository, *Heart Disease* dataset (ID 45): https://archive.ics.uci.edu/dataset/45/heart+disease
File used	`processed.cleveland.data` (Cleveland Clinic Foundation): the subset used by almost all published experiments
Size	303 instances, 13 input attributes + 1 output attribute (`num`)

Missing values	Marked with `?` (4 in `ca`, 2 in `thal`)

Input and Output

Inputs (the 4 selected Features / Attributes)

Input Feature/Attribute	Meaning	Possible Values
ChestPain (`cp`)	Type of chest pain	Typical Angina, Atypical Angina, Non-Anginal Pain, Asymptomatic
Slope (`slope`)	Slope of the ST segment of the ECG at peak exercise	Upsloping, Flat, Downsloping
MajorVessels (`ca`)	Number of major blood vessels coloured by fluoroscopy	Zero, One, Two, Three
Thal (`thal`)	Result of the thallium heart scan	Normal, Fixed Defect, Reversible Defect

Output (Prediction)

Output Attribute	Possible Values
HeartDisease	Yes, No

Why these 4 attributes?

- They are **categorical**, exactly like the Titanic attributes, so the same **Label Encoding** approach works without any change.
- They are clinically meaningful signs of coronary artery disease: chest-pain type, the ECG response at peak exercise, blocked vessels and the thallium scan result.
- Among all 70 possible combinations of 4 categorical attributes in this dataset, this combination gave the **highest 5-fold cross-validation accuracy (84.5%)** with the same label encoding and SVC settings.

Step 1: Import Libraries

> **Understanding:** The same libraries as the Titanic system are used, so the two systems can be compared step by step.

```
"""
Purpose of this code:
----------------------
Imports the same libraries used in the Titanic system: data handling,
label encoding, data splitting, the Support Vector Classifier, evaluation
metrics, PrettyTable for output, and plotting libraries.
"""
```

```
import numpy as np                          # numerical arrays
import pandas as pd                         # DataFrames (tables)
import pickle                               # save / load the trained model
from sklearn.model_selection import train_test_split   # split training / testing
data
from sklearn.preprocessing import LabelEncoder          # text -> numbers
from sklearn import svm                                 # Support Vector Machine
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
from prettytable import PrettyTable                     # neat output table
import matplotlib.pyplot as plt                         # plots
import seaborn as sns                                   # confusion-matrix heatmap
```

Step 2: Load Sample Data

> **Understanding:** The original UCI file has **no header row**, so the 14 column names are given from the dataset's documentation (`heart-disease.names`). Missing values are written as `?` in the file, so `na_values="?"` tells Pandas to read them as missing (`NaN`).

```
"""
Purpose of this section:
-------------------------
Loads the original Cleveland Heart Disease data downloaded from the UCI
Machine Learning Repository into a pandas DataFrame.
"""

# Column names from the UCI documentation (the file itself has no header)
column_names = ["age", "sex", "cp", "trestbps", "chol", "fbs", "restecg",
                "thalach", "exang", "oldpeak", "slope", "ca", "thal", "num"]

# Read the raw file; "?" means a missing value
raw_data = pd.read_csv("heart_disease/processed.cleveland.data",
names=column_names, na_values="?")

print("\n\nRaw Heart Disease Data (first 10 rows):")
print("========================================\n")
pd.set_option("display.max_columns", None)
print(raw_data.head(10))
```

```
Raw Heart Disease Data (first 10 rows):
========================================

    age  sex   cp  trestbps   chol  fbs  restecg  thalach  exang  oldpeak  \
0  63.0  1.0  1.0     145.0  233.0  1.0      2.0    150.0    0.0      2.3
1  67.0  1.0  4.0     160.0  286.0  0.0      2.0    108.0    1.0      1.5
2  67.0  1.0  4.0     120.0  229.0  0.0      2.0    129.0    1.0      2.6
3  37.0  1.0  3.0     130.0  250.0  0.0      0.0    187.0    0.0      3.5
4  41.0  0.0  2.0     130.0  204.0  0.0      2.0    172.0    0.0      1.4
5  56.0  1.0  2.0     120.0  236.0  0.0      0.0    178.0    0.0      0.8
6  62.0  0.0  4.0     140.0  268.0  0.0      2.0    160.0    0.0      3.6
7  57.0  0.0  4.0     120.0  354.0  0.0      0.0    163.0    1.0      0.6
8  63.0  1.0  4.0     130.0  254.0  0.0      2.0    147.0    0.0      1.4
9  53.0  1.0  4.0     140.0  203.0  1.0      2.0    155.0    1.0      3.1

   slope   ca  thal  num
0    3.0  0.0   6.0    0
1    2.0  3.0   3.0    2
2    2.0  2.0   7.0    1
3    3.0  0.0   3.0    0
4    1.0  0.0   3.0    0
5    1.0  0.0   3.0    0
6    3.0  2.0   3.0    3
7    1.0  0.0   3.0    0
8    2.0  1.0   7.0    2
9    3.0  0.0   7.0    1
```

Step 3: Understand and Pre-process Sample Data

Step 3.1: Understand Sample Data

> **Understanding:** Unlike the Titanic sample, this is real data, so we look at more than the column names: the number of instances, the missing values and the distribution of the output.

Attribute	Meaning	Attribute	Meaning
`age`	Age in years	`thalach`	Maximum heart rate achieved
`sex`	1 = male, 0 = female	`exang`	Exercise-induced angina (1 = yes, 0 = no)
`cp`	Chest pain type (1 to 4)	`oldpeak`	ST depression caused by exercise
`trestbps`	Resting blood pressure (mm Hg)	`slope`	Slope of the peak exercise ST segment
`chol`	Serum cholesterol (mg/dl)	`ca`	Major vessels coloured by fluoroscopy (0 to 3)
`fbs`	Fasting blood sugar > 120 mg/dl	`thal`	3 = normal, 6 = fixed defect, 7 = reversible defect
`restecg`	Resting ECG result (0 to 2)	`num`	**Output:** 0 = no disease, 1 to 4 = disease present

```
"""
Purpose of this section:
-------------------------
Helps us understand the dataset by printing:
```

```
1. The names of all attributes.
2. The number of instances.
3. The number of missing values in each attribute.
4. The distribution of the output attribute (num).
"""

print("\n\nAttributes in Sample Data:")
print("==========================\n")
print(raw_data.columns)

print("\n\nNumber of Instances in Sample Data:", raw_data["age"].count())
print("=========================================\n")

print("Missing Values in Each Attribute:")
print("=================================\n")
print(raw_data.isnull().sum())

print("\n\nDistribution of the Output Attribute (num):")
print("===========================================\n")
print(raw_data["num"].value_counts().sort_index())
```

```
Attributes in Sample Data:
==========================

Index(['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach',
       'exang', 'oldpeak', 'slope', 'ca', 'thal', 'num'],
      dtype='object')


Number of Instances in Sample Data: 303
=======================================

Missing Values in Each Attribute:
=================================

age         0
sex         0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalach     0
exang       0
oldpeak     0
slope       0
ca          4
thal        2
num         0
dtype: int64


Distribution of the Output Attribute (num):
===========================================

num
0    164
1     55
2     36
3     35
4     13
Name: count, dtype: int64
```

Step 3.2: Pre-process Sample Data

> **Understanding:** Two pre-processing steps are needed (the Titanic sample needed none):
>
> 1. **Remove missing values:** 6 rows contain a `?`. They are only 2% of the data, so they are deleted.
> 2. **Convert the output into two classes:** `num` has 5 values (0 to 4). Following the UCI documentation, 0 means *no heart disease* and 1, 2, 3, 4 all mean *heart disease present*. So `num` becomes `HeartDisease` with the values **No** and **Yes**, just like `Survived` in the Titanic data.

```
"""
Purpose of this section:
-------------------------
1. Removes the rows that contain missing values.
2. Converts the output attribute "num" (0 to 4) into "HeartDisease" (No / Yes).
"""

# 1. Remove rows with missing values
clean_data = raw_data.dropna().copy()
print("Instances before removing missing values:", len(raw_data))
print("Instances after removing missing values :", len(clean_data))

# 2. Output: 0 -> "No", 1 to 4 -> "Yes"
clean_data["HeartDisease"] = clean_data["num"].apply(lambda value: "No" if value
== 0 else "Yes")

print("\n\nHeartDisease Attribute after Conversion:")
print("=======================================\n")
print(clean_data["HeartDisease"].value_counts())
```

```
Instances before removing missing values: 303
Instances after removing missing values : 297


HeartDisease Attribute after Conversion:
=======================================

HeartDisease
No     160
Yes    137
Name: count, dtype: int64
```

Step 4: Feature Extraction

> **Understanding:** This is where the assignment's constraint is applied: only **4 input attributes** are selected out of 13. Their numeric codes are converted into readable text values (for example `cp = 4` becomes `Asymptomatic`), so the sample data looks exactly like the Titanic sample data. The result is saved as `heart_disease/sample-data.csv`.

```
"""
Purpose of this section:
-------------------------
1. Selects the 4 input attributes (cp, slope, ca, thal) and the output.
2. Converts their numeric codes into readable categories (as in the Titanic data).
3. Saves the sample data to a CSV file.
"""

# Code -> readable value, taken from the UCI documentation
chest_pain_names = {1: "Typical Angina", 2: "Atypical Angina", 3: "Non-Anginal
Pain", 4: "Asymptomatic"}
slope_names = {1: "Upsloping", 2: "Flat", 3: "Downsloping"}
major_vessels_names = {0: "Zero", 1: "One", 2: "Two", 3: "Three"}
thal_names = {3: "Normal", 6: "Fixed Defect", 7: "Reversible Defect"}
```

```
sample_data = pd.DataFrame({
    "ChestPain": clean_data["cp"].astype(int).map(chest_pain_names),
    "Slope": clean_data["slope"].astype(int).map(slope_names),
    "MajorVessels": clean_data["ca"].astype(int).map(major_vessels_names),
    "Thal": clean_data["thal"].astype(int).map(thal_names),
    "HeartDisease": clean_data["HeartDisease"],
}).reset_index(drop=True)

# Save the sample data (same role as titanic/sample-data.csv)
sample_data.to_csv(r"heart_disease/sample-data.csv", index=False, header=True)

print("\n\nSample Data (first 15 of", len(sample_data), "instances):")
print("================================================\n")
print(sample_data.head(15))
```

```
Sample Data (first 15 of 297 instances):
================================================

           ChestPain        Slope MajorVessels               Thal HeartDisease
0     Typical Angina  Downsloping         Zero       Fixed Defect           No
1       Asymptomatic         Flat        Three             Normal          Yes
2       Asymptomatic         Flat          Two  Reversible Defect          Yes
3   Non-Anginal Pain  Downsloping         Zero             Normal           No
4    Atypical Angina    Upsloping         Zero             Normal           No
5    Atypical Angina    Upsloping         Zero             Normal           No
6       Asymptomatic  Downsloping          Two             Normal          Yes
7       Asymptomatic    Upsloping         Zero             Normal           No
8       Asymptomatic         Flat          One  Reversible Defect          Yes
9       Asymptomatic  Downsloping         Zero  Reversible Defect          Yes
10      Asymptomatic         Flat         Zero       Fixed Defect           No
11   Atypical Angina         Flat         Zero             Normal           No
12  Non-Anginal Pain         Flat          One       Fixed Defect          Yes
13   Atypical Angina    Upsloping         Zero  Reversible Defect           No
14  Non-Anginal Pain    Upsloping         Zero  Reversible Defect           No
```

Step 5: Label Encoding the Sample Data (Input and Output is converted in Numeric Representation)

Step 5.1: Train the Label Encoder

> **Understanding:** One LabelEncoder is trained per attribute on its list of possible values. The table printed below shows the number given to every value (alphabetical order), which is needed to read the encoded data later.

```
"""
Purpose of this section:
-------------------------
Sets up and trains one Label Encoder for each attribute.
"""

# Possible values of each attribute
chest_pain = pd.DataFrame({"ChestPain": ["Typical Angina", "Atypical Angina",
```

```
"Non-Anginal Pain", "Asymptomatic"]})
slope = pd.DataFrame({"Slope": ["Upsloping", "Flat", "Downsloping"]})
major_vessels = pd.DataFrame({"MajorVessels": ["Zero", "One", "Two", "Three"]})
thal = pd.DataFrame({"Thal": ["Normal", "Fixed Defect", "Reversible Defect"]})
heart_disease = pd.DataFrame({"HeartDisease": ["Yes", "No"]})

# Create one LabelEncoder per attribute
chest_pain_label_encoder = LabelEncoder()
slope_label_encoder = LabelEncoder()
major_vessels_label_encoder = LabelEncoder()
thal_label_encoder = LabelEncoder()
heart_disease_label_encoder = LabelEncoder()

# Train (fit) each encoder on its possible values
chest_pain_label_encoder.fit(np.ravel(chest_pain))
slope_label_encoder.fit(np.ravel(slope))
major_vessels_label_encoder.fit(np.ravel(major_vessels))
thal_label_encoder.fit(np.ravel(thal))
heart_disease_label_encoder.fit(np.ravel(heart_disease))

# Show the number given to each value
encoders = {"ChestPain": chest_pain_label_encoder, "Slope": slope_label_encoder,
            "MajorVessels": major_vessels_label_encoder, "Thal":
thal_label_encoder,
            "HeartDisease": heart_disease_label_encoder}
codes_table = PrettyTable(["Attribute", "Value", "Code"])
for name, encoder in encoders.items():
    for code_number, value in enumerate(encoder.classes_):
        codes_table.add_row([name, value, code_number])
print(codes_table)
```

```
+--------------+-------------------+------+
|  Attribute   |       Value       | Code |
+--------------+-------------------+------+
|  ChestPain   |    Asymptomatic   |  0   |
|  ChestPain   |  Atypical Angina  |  1   |
|  ChestPain   |  Non-Anginal Pain |  2   |
|  ChestPain   |   Typical Angina  |  3   |
|    Slope     |    Downsloping    |  0   |
|    Slope     |        Flat       |  1   |
|    Slope     |     Upsloping     |  2   |
| MajorVessels |        One        |  0   |
| MajorVessels |       Three       |  1   |
| MajorVessels |        Two        |  2   |
| MajorVessels |        Zero       |  3   |
|     Thal     |    Fixed Defect   |  0   |
|     Thal     |       Normal      |  1   |
|     Thal     | Reversible Defect |  2   |
| HeartDisease |         No        |  0   |
| HeartDisease |        Yes        |  1   |
+--------------+-------------------+------+
```

Step 5.2: Label Encode the Output

> **Understanding:** `HeartDisease` is converted: `No = 0`, `Yes = 1`.

```
"""
Purpose of this section:
```

```
-------------------------
Applies Label Encoding to the output attribute "HeartDisease".
"""

sample_data_encoded_output = sample_data.copy()

print("\n\nHeartDisease Attribute After Label Encoding (first 10 rows):")
print("============================================================\n")
sample_data["encoded_heart_disease"] =
heart_disease_label_encoder.transform(sample_data["HeartDisease"])
print(sample_data[["HeartDisease", "encoded_heart_disease"]].head(10))

# Replace the output column with its encoded values
sample_data_encoded_output["HeartDisease"] = sample_data["encoded_heart_disease"]

sample_data_encoded_output.to_csv(r"heart_disease/sample-data-encoded-output.csv",
index=False, header=True)
print("\nSaved: heart_disease/sample-data-encoded-output.csv")
```

```
HeartDisease Attribute After Label Encoding (first 10 rows):
============================================================

  HeartDisease  encoded_heart_disease
0           No                      0
1          Yes                      1
2          Yes                      1
3           No                      0
4           No                      0
5           No                      0
6          Yes                      1
7           No                      0
8          Yes                      1
9          Yes                      1

Saved: heart_disease/sample-data-encoded-output.csv
```

Step 5.3: Label Encode the Input

> **Understanding:** The four input attributes are converted with their own encoders. After this step the whole table is numeric and ready for the algorithm.

```
"""
Purpose of this section:
-------------------------
Applies Label Encoding to the 4 input attributes.
"""

sample_data_encoded = sample_data_encoded_output.copy()

sample_data_encoded["ChestPain"] =
chest_pain_label_encoder.transform(sample_data_encoded_output["ChestPain"])
sample_data_encoded["Slope"] =
slope_label_encoder.transform(sample_data_encoded_output["Slope"])
sample_data_encoded["MajorVessels"] =
major_vessels_label_encoder.transform(sample_data_encoded_output["MajorVessels"])
sample_data_encoded["Thal"] =
```

```
thal_label_encoder.transform(sample_data_encoded_output["Thal"])

print("\n\nSample Data after Label Encoding (first 15 rows):")
print("================================================\n")
print(sample_data_encoded.head(15))

sample_data_encoded.to_csv(r"heart_disease/sample-data-encoded.csv", index=False,
header=True)
print("\nSaved: heart_disease/sample-data-encoded.csv")
```

```
Sample Data after Label Encoding (first 15 rows):
================================================

    ChestPain  Slope  MajorVessels  Thal  HeartDisease
0           3      0             3     0             0
1           0      1             1     1             1
2           0      1             2     2             1
3           2      0             3     1             0
4           1      2             3     1             0
5           1      2             3     1             0
6           0      0             2     1             1
7           0      2             3     1             0
8           0      1             0     2             1
9           0      0             3     2             1
10          0      1             3     0             0
11          1      1             3     1             0
12          2      1             0     0             1
13          1      2             3     2             0
14          2      2             3     2             0

Saved: heart_disease/sample-data-encoded.csv
```

Step 6: Execute the Training Phase

Step 6.1: Splitting Sample Data into Training Data and Testing Data

> **Understanding:** Same settings as the Titanic system: **80% training** (237 patients) and **20% testing** (60 patients), `random_state=0`, `shuffle=False`.

```
"""
Purpose of this section:
------------------------
Splits the dataset into Training Data (80%) and Testing Data (20%).
"""

training_data_encoded, testing_data_encoded = train_test_split(
    sample_data_encoded,
    test_size=0.2,
    random_state=0,
    shuffle=False
)

training_data_encoded.to_csv(r"heart_disease/training-data-encoded.csv",
index=False, header=True)
```

```
testing_data_encoded.to_csv(r"heart_disease/testing-data-encoded.csv",
index=False, header=True)

print("Training Data shape:", training_data_encoded.shape)
print("Testing Data shape:", testing_data_encoded.shape)
```

```
Training Data shape: (237, 5)
Testing Data shape: (60, 5)
```

Step 6.2: Splitting Input Vectors and Outputs / Labels of Training Data

> **Understanding:** First 4 columns = input vectors, last column = output label.

```
"""
Purpose of this section:
-------------------------
Separates the input vectors and the output labels of the training data.
"""

input_vector_train = training_data_encoded.iloc[:, :-1]
output_label_train = training_data_encoded.iloc[:, -1]

print("Input vectors (train) shape:", input_vector_train.shape)
print("Output labels (train) shape:", output_label_train.shape)
```

```
Input vectors (train) shape: (237, 4)
Output labels (train) shape: (237,)
```

Step 6.3: Train the Support Vector Classifier

> **Understanding:** The same algorithm with the same settings (`gamma='auto'` , `random_state=0`) learns the boundary between patients with and without heart disease.

```
"""
Purpose of this section:
-------------------------
Trains a Support Vector Classifier (SVC) on the training data.
"""

print("\n\nTraining the Support Vector Classifier on Training Data")
print("=========================================================\n")

svc_model = svm.SVC(gamma="auto", random_state=0)
svc_model.fit(input_vector_train, np.ravel(output_label_train))
print(svc_model)
```

```
Training the Support Vector Classifier on Training Data
=======================================================

SVC(gamma='auto', random_state=0)
```

Step 6.4: Save the Trained Model

> **Understanding:** The trained model is saved as `heart_disease/svc_trained_model.pkl`. This file is uploaded with the submission and used by the cloud app.

```
"""
Purpose of this section:
-------------------------
Saves the trained SVC model to a file so it can be reused without retraining.
"""

pickle.dump(svc_model, open("heart_disease/svc_trained_model.pkl", "wb"))
print("Saved: heart_disease/svc_trained_model.pkl")
```

```
Saved: heart_disease/svc_trained_model.pkl
```

Step 7: Execute the Testing Phase

Step 7.1: Splitting Input Vectors and Outputs/Labels of Testing Data

> **Understanding:** The 60 testing patients are split into inputs and true labels.

```
"""
Purpose of this section:
-------------------------
Separates the input vectors and the output labels of the testing data.
"""

input_vector_test = testing_data_encoded.iloc[:, :-1]
output_label_test = testing_data_encoded.iloc[:, -1]

print("Input vectors (test) shape:", input_vector_test.shape)
print("Output labels (test) shape:", output_label_test.shape)
```

```
Input vectors (test) shape: (60, 4)
Output labels (test) shape: (60,)
```

Step 7.2: Load the Saved Model

> **Understanding:** The model is loaded from the `.pkl` file to prove the saved file works.

```
"""
Purpose of this section:
-------------------------
Loads the saved SVC model from disk.
"""

model = pickle.load(open("heart_disease/svc_trained_model.pkl", "rb"))
print("Model loaded from heart_disease/svc_trained_model.pkl")
```

```
Model loaded from heart_disease/svc_trained_model.pkl
```

Step 7.3: Evaluate the Machine Learning Model

Step 7.3.1: Make Predictions with the Trained Models on Testing Data

> **Understanding:** The model predicts the 60 unseen patients. The first 15 predictions are shown next to the true labels; all 60 are saved to `model-predictions.csv`.

```
"""
Purpose of this section:
-------------------------
Uses the trained model to predict the testing data.
"""

model_predictions = model.predict(input_vector_test)

pd.options.mode.chained_assignment = None
testing_data_encoded["Predictions"] = model_predictions
testing_data_encoded.to_csv(r"heart_disease/model-predictions.csv", index=False,
header=True)

model_predictions = testing_data_encoded
print("\n\nPredictions Returned by svc_trained_model (first 15 of",
len(model_predictions), "):")
print("=========================================================================\n")
print(model_predictions.head(15))
```

```
Predictions Returned by svc_trained_model (first 15 of 60 ):
=========================================================================

     ChestPain  Slope  MajorVessels  Thal  HeartDisease  Predictions
237          1      2             3     1             0            0
238          1      2             3     1             0            0
239          0      2             3     1             0            0
240          3      1             2     1             1            0
241          2      2             3     1             0            0
242          0      1             3     1             1            0
243          0      2             0     2             1            1
244          0      1             0     1             1            1
245          0      2             2     2             1            1
246          1      2             3     1             0            0
247          0      1             3     0             0            0
248          0      1             0     2             1            1
249          0      1             0     2             0            1
250          2      2             3     1             0            0
251          0      1             3     1             0            0
```

Step 7.4: Calculate the Accuracy Score

> **Understanding:** Accuracy = correct predictions / 60 testing patients.

```
"""
Purpose of this section:
-------------------------
Calculates the accuracy of the model on the testing data.
"""
```

```
model_accuracy_score = accuracy_score(
    model_predictions["HeartDisease"],
    model_predictions["Predictions"]
)

print("\n\nAccuracy Score:")
print("===============\n")
print(round(model_accuracy_score, 2))
print("Correct predictions:", (model_predictions["HeartDisease"] ==
model_predictions["Predictions"]).sum(),
      "out of", len(model_predictions))
```

```
Accuracy Score:
===============

0.75
Correct predictions: 45 out of 60
```

Step 7.5: Detailed Evaluation Metrics (Precision, Recall, F1, Confusion Matrix)

> **Understanding:** In medicine, **Recall for the "Yes" class** is especially important: it shows how many patients who really have heart disease the model catches. A missed patient (false negative) is more dangerous than a false alarm.

```
"""
Purpose of this section:
------------------------
Prints precision, recall, F1-score and the confusion matrix.
"""

print("\nClassification Report:")
print("======================\n")
print(classification_report(
    model_predictions["HeartDisease"],
    model_predictions["Predictions"],
    target_names=["No Heart Disease (0)", "Heart Disease (1)"]
))

conf_matrix = confusion_matrix(model_predictions["HeartDisease"],
model_predictions["Predictions"])
conf_matrix_df = pd.DataFrame(
    conf_matrix,
    index=["Actual: No", "Actual: Yes"],
    columns=["Predicted: No", "Predicted: Yes"]
)
print("\nConfusion Matrix:")
print("=================\n")
print(conf_matrix_df)

plt.figure(figsize=(5, 4))
sns.heatmap(conf_matrix, annot=True, fmt="d", cmap="Blues",
            xticklabels=["No", "Yes"], yticklabels=["No", "Yes"])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - SVC on Heart Disease Testing Data")
```

```
plt.tight_layout()
plt.show()
```

```
Classification Report:
======================

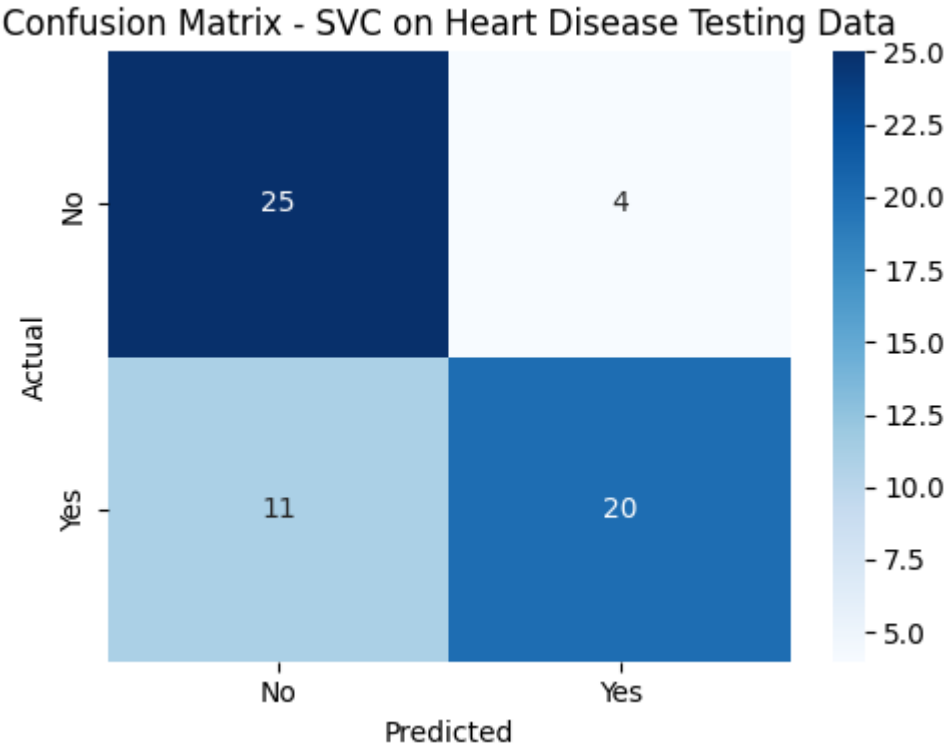
                      precision    recall  f1-score   support

No Heart Disease (0)       0.69      0.86      0.77        29
   Heart Disease (1)       0.83      0.65      0.73        31

            accuracy                           0.75        60
           macro avg       0.76      0.75      0.75        60
        weighted avg       0.77      0.75      0.75        60


Confusion Matrix:
=================

             Predicted: No  Predicted: Yes
Actual: No              25               4
Actual: Yes             11              20
```



Step 8: Execute the Application Phase

Step 8.1: Take Input from User

> **Understanding:** A new patient is entered. As in the Titanic notebook, the values are pre-filled so the notebook runs from top to bottom; the `input()` version is given in the comment.

```
"""
Purpose of this section:
------------------------
Takes a new patient's details. To type your own values, replace the four lines
below with:
    chest_pain_input = input("Chest Pain (Typical Angina, Atypical Angina, Non-
Anginal Pain, Asymptomatic): ").strip()
    slope_input = input("Slope (Upsloping, Flat, Downsloping): ").strip()
    major_vessels_input = input("Major Vessels (Zero, One, Two, Three): ").strip()
    thal_input = input("Thal (Normal, Fixed Defect, Reversible Defect): ").strip()
"""

# Demo patient
chest_pain_input = "Asymptomatic"
slope_input = "Flat"
major_vessels_input = "Two"
thal_input = "Reversible Defect"

print(f"ChestPain: {chest_pain_input}, Slope: {slope_input}, "
      f"MajorVessels: {major_vessels_input}, Thal: {thal_input}")
```

```
ChestPain: Asymptomatic, Slope: Flat, MajorVessels: Two, Thal: Reversible Defect
```

Step 8.2: Convert User Input into Feature Vector (Exactly Same as Feature Vectors of Sample Data)

> **Understanding:** Same column names and order as the training data.

```
"""
Purpose of this section:
------------------------
Converts the user's input into a one-row DataFrame (feature vector).
"""

user_input = pd.DataFrame({
    "ChestPain": [chest_pain_input],
    "Slope": [slope_input],
    "MajorVessels": [major_vessels_input],
    "Thal": [thal_input]
})

print("\n\nUser Input Feature Vector:")
print("==========================\n")
print(user_input)
```

```
User Input Feature Vector:
==========================

      ChestPain Slope MajorVessels              Thal
0  Asymptomatic  Flat          Two  Reversible Defect
```

Step 8.3: Label Encoding of Feature Vector (Exactly Same as Label Encoded Feature Vectors of Sample Data)

> **Understanding:** The same trained encoders turn the text into the same numbers the model learned from.

```
"""
Purpose of this section:
-------------------------
Encodes the user's input with the previously trained LabelEncoders.
"""

unseen_data_features = user_input.copy()

unseen_data_features["ChestPain"] =
chest_pain_label_encoder.transform(user_input["ChestPain"])
unseen_data_features["Slope"] = slope_label_encoder.transform(user_input["Slope"])
unseen_data_features["MajorVessels"] =
major_vessels_label_encoder.transform(user_input["MajorVessels"])
unseen_data_features["Thal"] = thal_label_encoder.transform(user_input["Thal"])

print("\n\nUser Input Encoded Feature Vector:")
print("==================================\n")
print(unseen_data_features)
```

```
User Input Encoded Feature Vector:
==================================

   ChestPain  Slope  MajorVessels  Thal
0          0      1             2     2
```

Step 8.4: Load the Saved Model

```
"""
Purpose of this section:
-------------------------
Loads the saved SVC model for the Application Phase.
"""

model = pickle.load(open("heart_disease/svc_trained_model.pkl", "rb"))
```

Step 8.5: Model Prediction

Step 8.5.1: Apply Model on the Label Encoded Feature Vector of unseen instance and return Prediction to the User

> **Understanding:** `1` means heart disease is present, `0` means it is not.

```
"""
Purpose of this section:
-------------------------
Predicts whether the new patient has heart disease.
"""

predicted_heart_disease = model.predict(unseen_data_features)
```

```
if predicted_heart_disease == 1:
    prediction = "HEART DISEASE PRESENT"
if predicted_heart_disease == 0:
    prediction = "NO HEART DISEASE"

pretty_table = PrettyTable()
pretty_table.add_column("       ** Prediction **       ", [prediction])

print(pretty_table)
```

```
+--------------------------------+
|        ** Prediction **        |
+--------------------------------+
|     HEART DISEASE PRESENT      |
+--------------------------------+
```

Step 9: Execute the Feedback Phase

Step 9.1: Collect Feedback from Users and Domain Experts on Performance of the Model Deployed in the Real World

> **Understanding:** Example feedback that a cardiologist and test users could give after trying the deployed app.

```
"""
Purpose of this section:
-------------------------
Example feedback on the model's predictions.
"""

example_feedback = pd.DataFrame({
    "Reviewer": ["Domain Expert (Cardiologist)", "Domain Expert (Cardiologist)",
"Student Tester"],
    "Case": [
        "Asymptomatic chest pain, flat ST slope, 2 vessels, reversible defect ->
predicted PRESENT",
        "Non-anginal pain, upsloping ST slope, 0 vessels, normal thal -> predicted
NO",
        "Typical angina, upsloping ST slope, 0 vessels, normal thal -> predicted
NO",
    ],
    "Feedback": [
        "Correct: this combination is a classic high-risk pattern.",
        "Correct: all four signs are reassuring.",
        "Model uses only 4 attributes, so age, sex, blood pressure, cholesterol
and exercise angina are ignored; "
        "a doctor would still order further tests.",
    ],
})
for i, row in example_feedback.iterrows():
    print(f"{i + 1}. {row['Reviewer']}")
    print(f"   Case    : {row['Case']}")
    print(f"   Feedback: {row['Feedback']}\n")
```

```
1. Domain Expert (Cardiologist)
   Case    : Asymptomatic chest pain, flat ST slope, 2 vessels, reversible defect -
> predicted PRESENT
   Feedback: Correct: this combination is a classic high-risk pattern.

2. Domain Expert (Cardiologist)
   Case    : Non-anginal pain, upsloping ST slope, 0 vessels, normal thal -> predic
ted NO
   Feedback: Correct: all four signs are reassuring.

3. Student Tester
   Case    : Typical angina, upsloping ST slope, 0 vessels, normal thal -> predicte
d NO
   Feedback: Model uses only 4 attributes, so age, sex, blood pressure, cholesterol
and exercise angina are ignored; a doctor would still order further tests.
```

Step 9.2: Make a List of Potential Improvements

```
"""
Purpose of this section:
-------------------------
Potential improvements identified from the feedback above.
"""

improvements = [
    "Add more attributes (age, sex, exercise angina, maximum heart rate, oldpeak)
once the 4-attribute limit is removed",
    "Shuffle and stratify the train-test split, and use k-fold cross validation
for a more reliable accuracy",
    "Tune the SVC (C, gamma, kernel) and compare it with Logistic Regression and
Random Forest",
    "Prefer higher Recall for the 'Yes' class, because missing a sick patient is
worse than a false alarm",
    "Combine the Cleveland data with the Hungarian, Swiss and VA data in the same
UCI dataset to get more training examples",
]
for i, item in enumerate(improvements, 1):
    print(f"{i}. {item}")
```

```
1. Add more attributes (age, sex, exercise angina, maximum heart rate, oldpeak) onc
e the 4-attribute limit is removed
2. Shuffle and stratify the train-test split, and use k-fold cross validation for a
more reliable accuracy
3. Tune the SVC (C, gamma, kernel) and compare it with Logistic Regression and Rand
om Forest
4. Prefer higher Recall for the 'Yes' class, because missing a sick patient is wors
e than a false alarm
5. Combine the Cleveland data with the Hungarian, Swiss and VA data in the same UCI
dataset to get more training examples
```

Step 9.3: Improve the Model Based on Feedback

```
"""
Purpose of this section:
-------------------------
In a real deployment, the next version would apply the improvements above,
then re-run Steps 6 and 7 to confirm the accuracy and recall improve
```

```
before replacing the deployed model.
"""

print("Next version: add numeric attributes, use stratified cross validation and
tuned SVC, "
      "then re-evaluate before replacing heart_disease/svc_trained_model.pkl")
```

```
Next version: add numeric attributes, use stratified cross validation and tuned SV
C, then re-evaluate before replacing heart_disease/svc_trained_model.pkl
```

Task 2 Result

The Heart Disease Prediction System was built by following **exactly the same steps** as the Titanic system, with real pre-processing (missing values removed, a 5-class output converted to Yes/No) and feature extraction (4 of 13 attributes selected). The SVC reached an **accuracy of 0.75** on the 60 held-out patients (45 correct). This is lower than its 5-fold cross-validation accuracy of 84.5% because, with `shuffle=False` (kept from the Titanic code), the testing data is simply the last 60 rows of the file. Its recall for the *Yes* class is lower than for *No*, which Step 9 lists as an improvement. The model is saved as `heart_disease/svc_trained_model.pkl` and deployed in Part C.

PART C: Step 10, Deploying Both Models to the Cloud

Both trained models are served by **one Streamlit web app** with two tabs, deployed on **Streamlit Community Cloud** (free), the same platform used for the instructor's live Titanic demo. Anyone can open the link in a browser; nothing has to be installed.

Step 10.1: Copy the Trained Models into the App Folder

> **Understanding:** The web app needs the `.pkl` files that the notebook produced. They are copied into `streamlit-app/` with clear names.

```
"""
Purpose of this section:
-------------------------
Copies both trained models into the streamlit-app folder used for deployment.
"""

import shutil

shutil.copy("titanic/svc_trained_model.pkl", "streamlit-
app/titanic_svc_trained_model.pkl")
shutil.copy("heart_disease/svc_trained_model.pkl", "streamlit-
app/heart_svc_trained_model.pkl")
```

```
print("Copied: streamlit-app/titanic_svc_trained_model.pkl")
print("Copied: streamlit-app/heart_svc_trained_model.pkl")
```

```
Copied: streamlit-app/titanic_svc_trained_model.pkl
Copied: streamlit-app/heart_svc_trained_model.pkl
```

Step 10.2: Check that the App Gives the Same Predictions as the Notebook

> **Understanding:** The app recreates the LabelEncoders from the same fixed value lists. This cell repeats that and confirms the deployed models give the same answers as Steps 8.5 of Part A and Part B.

```
"""
Purpose of this section:
------------------------
Loads the copied models exactly as the web app does and predicts the two demo
inputs.
"""

titanic_app_model = pickle.load(open("streamlit-
app/titanic_svc_trained_model.pkl", "rb"))
heart_app_model = pickle.load(open("streamlit-app/heart_svc_trained_model.pkl",
"rb"))

titanic_demo = pd.DataFrame({
    "PClass": LabelEncoder().fit(["First", "Second",
"Third"]).transform(["First"]),
    "Gender": LabelEncoder().fit(["Male", "Female"]).transform(["Male"]),
    "Sibling": LabelEncoder().fit(["Zero", "One", "Two",
"Three"]).transform(["Zero"]),
    "Embarked": LabelEncoder().fit(["Southampton", "Cherbourg",
"Queenstown"]).transform(["Queenstown"]),
})
heart_demo = pd.DataFrame({
    "ChestPain": LabelEncoder().fit(["Typical Angina", "Atypical Angina", "Non-
Anginal Pain", "Asymptomatic"]).transform(["Asymptomatic"]),
    "Slope": LabelEncoder().fit(["Upsloping", "Flat",
"Downsloping"]).transform(["Flat"]),
    "MajorVessels": LabelEncoder().fit(["Zero", "One", "Two",
"Three"]).transform(["Two"]),
    "Thal": LabelEncoder().fit(["Normal", "Fixed Defect", "Reversible
Defect"]).transform(["Reversible Defect"]),
})

titanic_result = "SURVIVED" if titanic_app_model.predict(titanic_demo)[0] == 1
else "NOT SURVIVED"
heart_result = "HEART DISEASE PRESENT" if heart_app_model.predict(heart_demo)[0]
== 1 else "NO HEART DISEASE"

print("Titanic app       | First, Male, Zero, Queenstown              ->",
titanic_result)
print("Heart Disease app | Asymptomatic, Flat, Two, Reversible Defect ->",
heart_result)
```

```
Titanic app       | First, Male, Zero, Queenstown              -> SURVIVED
Heart Disease app | Asymptomatic, Flat, Two, Reversible Defect -> HEART DISEASE PRE
SENT
```

Step 10.3: The Web App (`streamlit-app/app.py`)

> **Understanding:** The app (1) loads each `.pkl` model, (2) recreates the LabelEncoders, (3) shows drop-down menus for the 4 inputs, and (4) calls `model.predict()` when the button is clicked. Its `requirements.txt` pins the same library versions used to train the models, so the saved models load correctly on the cloud.

```
"""
AIC354 Assignment 2: End-to-End Machine Learning System
Zain Aftab (FA24-BSE-155), COMSATS University Islamabad, Lahore Campus
-------------------------------------------------------------------------
-
One Streamlit web app that serves both trained models:
  Tab 1: Titanic Passenger Survival Prediction (Support Vector
Classifier)
  Tab 2: Heart Disease Prediction (Support Vector Classifier)

Each tab loads the .pkl model saved by the notebook and recreates the
same LabelEncoders (fitted on the same fixed category lists), so the
predictions match the notebook exactly.
"""

import pickle
from pathlib import Path

import pandas as pd
import streamlit as st
from sklearn.preprocessing import LabelEncoder

st.set_page_config(page_title="AIC354 ML Prediction Systems",
page_icon="🩺", layout="centered")

APP_DIR = Path(__file__).parent


@st.cache_resource
def load_model(file_name):
    with open(APP_DIR / file_name, "rb") as f:
        return pickle.load(f)


# Same category lists used to train the LabelEncoders in the notebook
TITANIC_CATEGORIES = {
    "PClass": ["First", "Second", "Third"],
    "Gender": ["Male", "Female"],
    "Sibling": ["Zero", "One", "Two", "Three"],
    "Embarked": ["Southampton", "Cherbourg", "Queenstown"],
```

```
}

HEART_CATEGORIES = {
    "ChestPain": ["Typical Angina", "Atypical Angina", "Non-Anginal
Pain", "Asymptomatic"],
    "Slope": ["Upsloping", "Flat", "Downsloping"],
    "MajorVessels": ["Zero", "One", "Two", "Three"],
    "Thal": ["Normal", "Fixed Defect", "Reversible Defect"],
}


@st.cache_resource
def build_encoders(categories):
    encoders = {}
    for column, values in categories.items():
        encoders[column] = LabelEncoder().fit(values)
    return encoders


def encode(user_input, encoders):
    encoded = user_input.copy()
    for column in user_input.columns:
        encoded[column] =
encoders[column].transform(user_input[column])
    return encoded


st.title("Machine Learning Prediction Systems")
st.markdown(
    "**AIC354 Machine Learning Fundamentals, Assignment 2**  \n"
    "Zain Aftab (FA24-BSE-155), COMSATS University Islamabad, Lahore
Campus"
)

titanic_tab, heart_tab = st.tabs(["🚢 Titanic Survival", "🩺 Heart
Disease"])

# ----------------------------------------------------------------------
-------
# Tab 1: Titanic Passenger Survival Prediction
# ----------------------------------------------------------------------
-------
with titanic_tab:
    st.subheader("Titanic Passenger Survival Prediction")
    st.write("Pick a passenger's details. The trained Support Vector
Classifier predicts whether they survived.")

    col1, col2 = st.columns(2)
    with col1:
        pclass = st.selectbox("Passenger Class (PClass)",
TITANIC_CATEGORIES["PClass"], index=2)
```

```
        gender = st.selectbox("Gender", TITANIC_CATEGORIES["Gender"])
    with col2:
        sibling = st.selectbox("Siblings/Spouses Aboard",
TITANIC_CATEGORIES["Sibling"])
        embarked = st.selectbox("Port of Embarkation",
TITANIC_CATEGORIES["Embarked"])

    if st.button("Predict Survival", type="primary",
use_container_width=True):
        user_input = pd.DataFrame({
            "PClass": [pclass], "Gender": [gender], "Sibling":
[sibling], "Embarked": [embarked],
        })
        encoded = encode(user_input,
build_encoders(TITANIC_CATEGORIES))
        prediction =
load_model("titanic_svc_trained_model.pkl").predict(encoded)[0]
        if prediction == 1:
            st.success("### ✅ SURVIVED")
        else:
            st.error("### ❌ NOT SURVIVED")
        with st.expander("Encoded feature vector sent to the model"):
            st.dataframe(encoded, hide_index=True)

# ----------------------------------------------------------------------
-------
# Tab 2: Heart Disease Prediction
# ----------------------------------------------------------------------
-------
with heart_tab:
    st.subheader("Heart Disease Prediction")
    st.write("Enter a patient's details. The trained Support Vector
Classifier predicts whether heart disease is present.")

    col1, col2 = st.columns(2)
    with col1:
        chest_pain = st.selectbox("Chest Pain Type",
HEART_CATEGORIES["ChestPain"], index=3)
        slope = st.selectbox("Slope of Peak Exercise ST Segment",
HEART_CATEGORIES["Slope"], index=1)
    with col2:
        major_vessels = st.selectbox("Major Vessels Coloured by
Fluoroscopy", HEART_CATEGORIES["MajorVessels"])
        thal = st.selectbox("Thal (Thallium Stress Test)",
HEART_CATEGORIES["Thal"])

    if st.button("Predict Heart Disease", type="primary",
use_container_width=True):
        user_input = pd.DataFrame({
            "ChestPain": [chest_pain], "Slope": [slope],
            "MajorVessels": [major_vessels], "Thal": [thal],
```

```
        })
        encoded = encode(user_input, build_encoders(HEART_CATEGORIES))
        prediction =
load_model("heart_svc_trained_model.pkl").predict(encoded)[0]
        if prediction == 1:
            st.error("### ⚠️ HEART DISEASE PRESENT")
        else:
            st.success("### ✅ NO HEART DISEASE")
        with st.expander("Encoded feature vector sent to the model"):
            st.dataframe(encoded, hide_index=True)

st.divider()
st.caption(
    "Built for teaching purposes. Titanic: 100-row simplified dataset.
"
    "Heart Disease: UCI Cleveland dataset (297 complete records, 4
input attributes). "
    "Not a medical diagnosis tool."
)
```

`streamlit-app/requirements.txt`

```
streamlit==1.64.0
scikit-learn==1.7.2
pandas==2.3.3
numpy==2.2.6
```

Step 10.4: Test the App Locally

```
pip install -r streamlit-app/requirements.txt
streamlit run streamlit-app/app.py
```

The app opens at `http://localhost:8501`. Try both tabs before deploying.

Step 10.5: Upload the Project to GitHub

1. Create a **public** repository on https://github.com (for example `aic354-ml-systems`).
2. Upload this project folder (**Add file → Upload files**). At minimum, the `streamlit-app/` folder with `app.py`, `requirements.txt` and both `.pkl` files.

Step 10.6: Deploy on Streamlit Community Cloud

1. Go to https://share.streamlit.io and **sign in with GitHub**.
2. Click **Create app → Deploy a public app from GitHub**.
3. Repository: `<your-username>/aic354-ml-systems`, Branch: `main`, Main file path: `streamlit-app/app.py`.
4. Open **Advanced settings → Python version** and choose **3.11** (the instructor's notes warn that the newest Python may not have scikit-learn wheels yet).

5. Click **Deploy**. After a minute or two the app is live at a public `https://<name>.streamlit.app` link.

Step 10.7: Share the Live Link

The live link is placed in the **first cell of this notebook**, as the submission guidelines require.

Conclusion and References

Comparison of the Two Systems

Aspect	Task 1: Titanic Survival	Task 2: Heart Disease
Problem type	Binary classification	Binary classification
Data source	Instructor's 100-row sample	UCI Heart Disease, Cleveland (303 rows)
Pre-processing	Not needed	6 rows with missing values removed; output 0 to 4 converted to No/Yes
Feature extraction	4 attributes given	4 of 13 attributes selected (ChestPain, Slope, MajorVessels, Thal)
Instances used	100 (80 train / 20 test)	297 (237 train / 60 test)
Encoding	LabelEncoder	LabelEncoder
Algorithm	SVC (`gamma='auto'`)	SVC (`gamma='auto'`)
Test accuracy	0.80	0.75
Saved model	`titanic/svc_trained_model.pkl`	`heart_disease/svc_trained_model.pkl`
Deployment	Streamlit Cloud (Titanic tab)	Streamlit Cloud (Heart Disease tab)

Conclusion

This assignment followed one Machine Learning workflow from start to finish twice. In **Task 1** the instructor's Titanic system was executed and every step was explained. In **Task 2** the same nine steps were reused on a real medical dataset: the data was downloaded from UCI, cleaned, reduced to four input attributes, label encoded, used to train and test a Support Vector Classifier, applied to a new patient and reviewed in a feedback phase. Finally, both models were placed in one web app for cloud deployment.

The main lesson is that the **ML cycle stays the same** for very different problems; what changes is the data understanding, pre-processing and feature selection, which is where most of the real work is.

References

1. Dr. Rao Muhammad Adeel Nawab and Mr. Mohsin Afzal, *Titanic Passenger Survival Prediction System* (Titanic_Passenger_Survival_Recommended.ipynb), https://github.com/haqnawaz99/ml-teaching-notebooks
2. Janosi, A., Steinbrunn, W., Pfisterer, M. and Detrano, R. (1989). *Heart Disease* [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C52P4X
3. scikit-learn developers, *Support Vector Machines* and *LabelEncoder* documentation, https://scikit-learn.org/stable/
4. The pandas development team, *pandas Documentation*, https://pandas.pydata.org/docs/
5. Streamlit, *Deploy your app on Community Cloud*, https://docs.streamlit.io/deploy/streamlit-community-cloud

---

Zain Aftab · FA24-BSE-155 · Section B

Machine Learning Fundamentals (AIC354) · Dr. Rao Muhammad Adeel Nawab

COMSATS University Islamabad, Lahore Campus